

# **API Security Risk Analysis Report**

## **Assessment of JSONPlaceholder Test API**

Prepared by: Kenny Ntsako Kosa  
Cyber Security Intern - Future Interns  
Date: June 26, 2026

# Executive Summary

This report presents a security risk analysis of the JSONPlaceholder API ([jsonplaceholder.typicode.com](https://jsonplaceholder.typicode.com)), a widely used mock API for testing and prototyping. The assessment focused on authentication, authorization, data exposure, and information disclosure—critical components of modern SaaS security.

Key findings include: unauthenticated access to all user data, broken object-level authorization (IDOR) allowing access to any user's profile, excessive data exposure where the API returns massive datasets without pagination, and server information disclosure revealing the underlying technology stack.

Immediate remediation steps include implementing API key/token requirements, enforcing user-specific access controls, implementing pagination, and obfuscating server headers to reduce the attack surface.

# Scope & Methodology

## API Tested:

- <https://jsonplaceholder.typicode.com> – Mock data service

## Scope of Assessment:

- Read-only GET requests
- No active exploitation or DoS testing
- Header and response inspection only

## Tools Used:

- Browser (Chrome) – Request/Response Inspection
- Canva – Report Design

**Ethics Statement:** All tests were conducted strictly within ethical guidelines on publicly available test APIs. No production systems or sensitive data were accessed.

# Risk Classification of Identified Vulnerabilities

| # | Endpoint         | Risk Identified                                                         | Severity                                                                                     | Business Impact                                                                                                                    | Remediation                                                                         |
|---|------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| 1 | GET /users       | Unauthenticated Access – No API key/token required.                     |  High     | Attackers can retrieve the entire user database (names, emails, addresses) without any authentication, leading to mass data theft. | Require a valid API key or authentication token for all endpoints.                  |
| 2 | GET /users/{id}  | Broken Object Level Authorization (IDOR) – Accessing other users' data. |  High   | By changing the ID in the URL (e.g., /users/1, /users/2), an attacker can view ANY user's personal details.                        | Implement server-side access controls. Users must only access their own data.       |
| 3 | GET /users       | Excessive Data Exposure – Returns all records at once.                  |  Medium | The API returns 100+ records in a single request, increasing data leakage risk and server load.                                    | Implement pagination (e.g., ?_limit=10&_page=1) to limit data returned per request. |
| 4 | Response Headers | Server Information Disclosure – Reveals tech stack.                     |  Low    | Headers reveal Express and heroku. Attackers can look for known exploits targeting these specific technologies.                    | Obfuscate or remove X-Powered-By header and customize the Server header.            |

# Remediation Strategy

## Remediation Roadmap (Priority Order)

### Priority 1 – Immediate Action (High Risk)

- Implement Authentication: Enforce API keys or OAuth2 tokens for all /users endpoints to prevent public scraping.
- Enforce Authorization: Add server-side logic so that a user with ID 1 cannot request the data of user 2 (IDOR prevention).

### Priority 2 – Short-Term Action (Medium Risk)

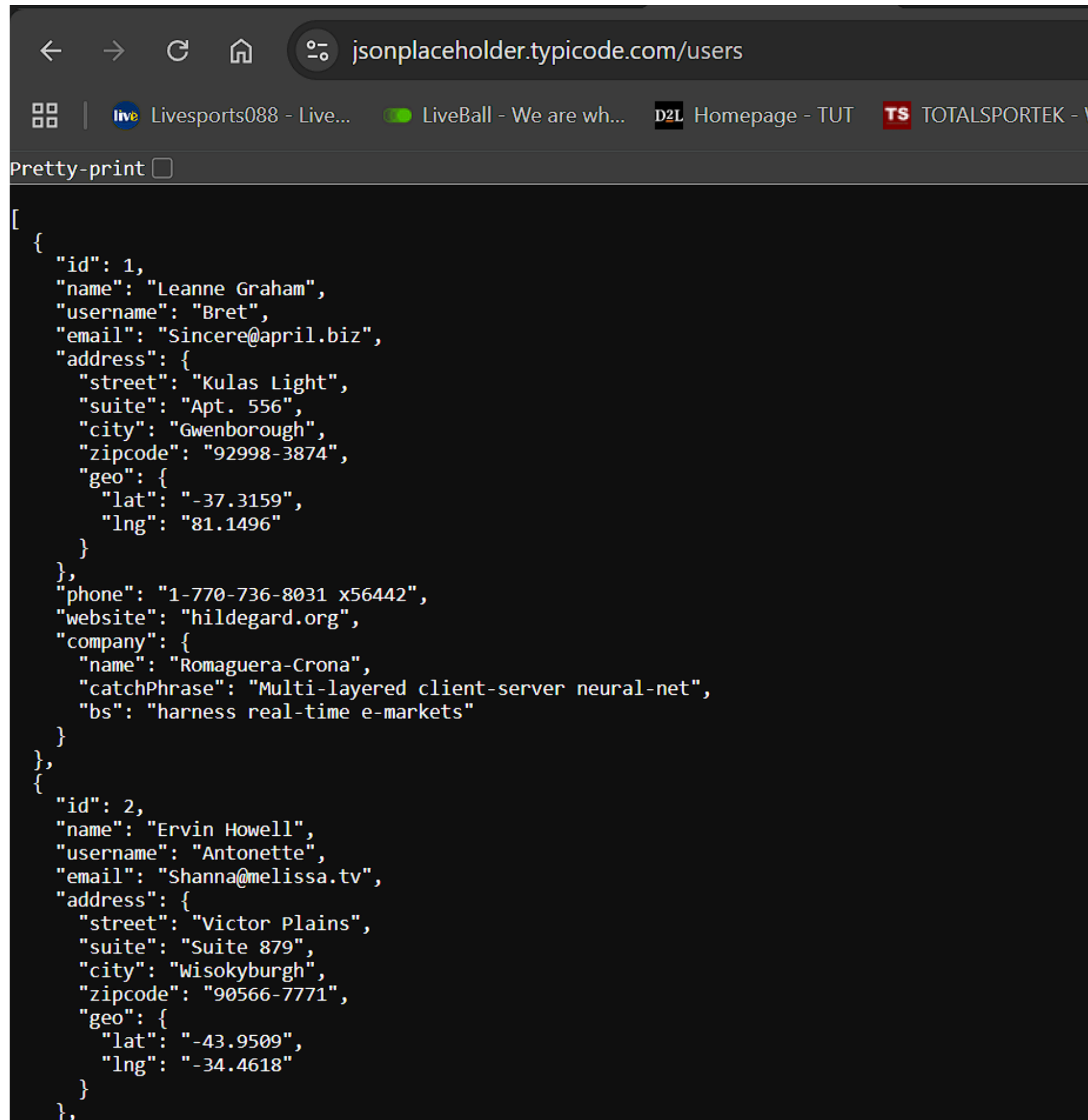
- Add Pagination: Modify /posts to accept \_limit and \_page parameters to prevent massive data dumps.

### Priority 3 – Low-Risk Action

- Obfuscate Server Headers: Remove or customize the X-Powered-By and Server headers to hide the technology stack from attackers.

# Figure 1: Unauthenticated access to the full user list.

## No API key or token required.

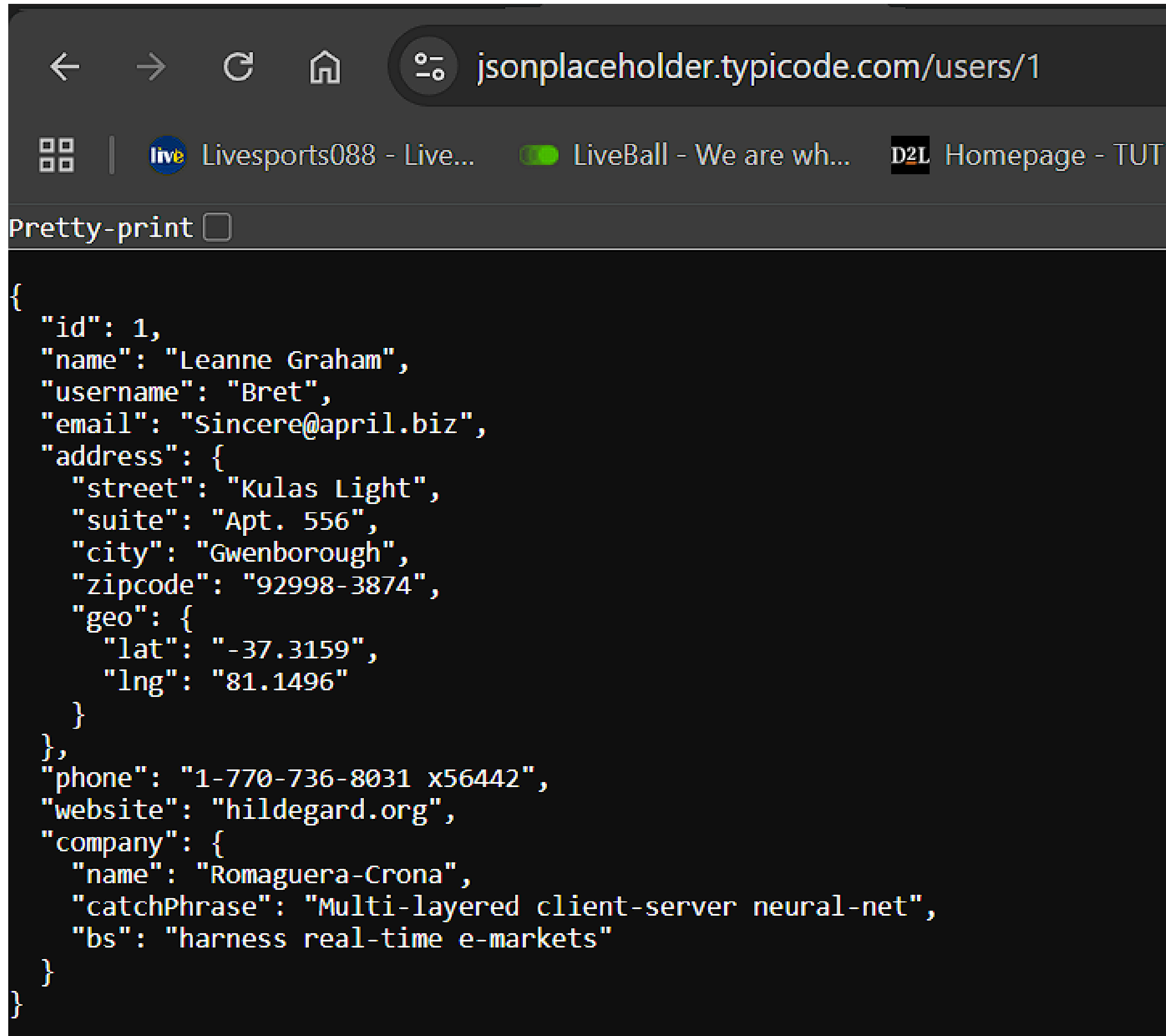


The screenshot shows a web browser window with the address bar displaying `jsonplaceholder.typicode.com/users`. The browser's tab bar shows several open tabs, including "Livesports088 - Live...", "LiveBall - We are wh...", "D2L Homepage - TUT", and "TOTALSPORTEK - V". Below the address bar, there is a "Pretty-print" checkbox. The main content area displays a JSON array of two user objects. The first user has an ID of 1, name "Leanne Graham", username "Bret", email "Sincere@april.biz", and a detailed address in Gwenborough. The second user has an ID of 2, name "Ervin Howell", username "Antonette", email "Shanna@melissa.tv", and a detailed address in Wisokyburgh.

```
[
  {
    "id": 1,
    "name": "Leanne Graham",
    "username": "Bret",
    "email": "Sincere@april.biz",
    "address": {
      "street": "Kulas Light",
      "suite": "Apt. 556",
      "city": "Gwenborough",
      "zipcode": "92998-3874",
      "geo": {
        "lat": "-37.3159",
        "lng": "81.1496"
      }
    },
    "phone": "1-770-736-8031 x56442",
    "website": "hildegard.org",
    "company": {
      "name": "Romaguera-Crona",
      "catchPhrase": "Multi-layered client-server neural-net",
      "bs": "harness real-time e-markets"
    }
  },
  {
    "id": 2,
    "name": "Ervin Howell",
    "username": "Antonette",
    "email": "Shanna@melissa.tv",
    "address": {
      "street": "Victor Plains",
      "suite": "Suite 879",
      "city": "Wisokyburgh",
      "zipcode": "90566-7771",
      "geo": {
        "lat": "-43.9509",
        "lng": "-34.4618"
      }
    }
  }
],
```



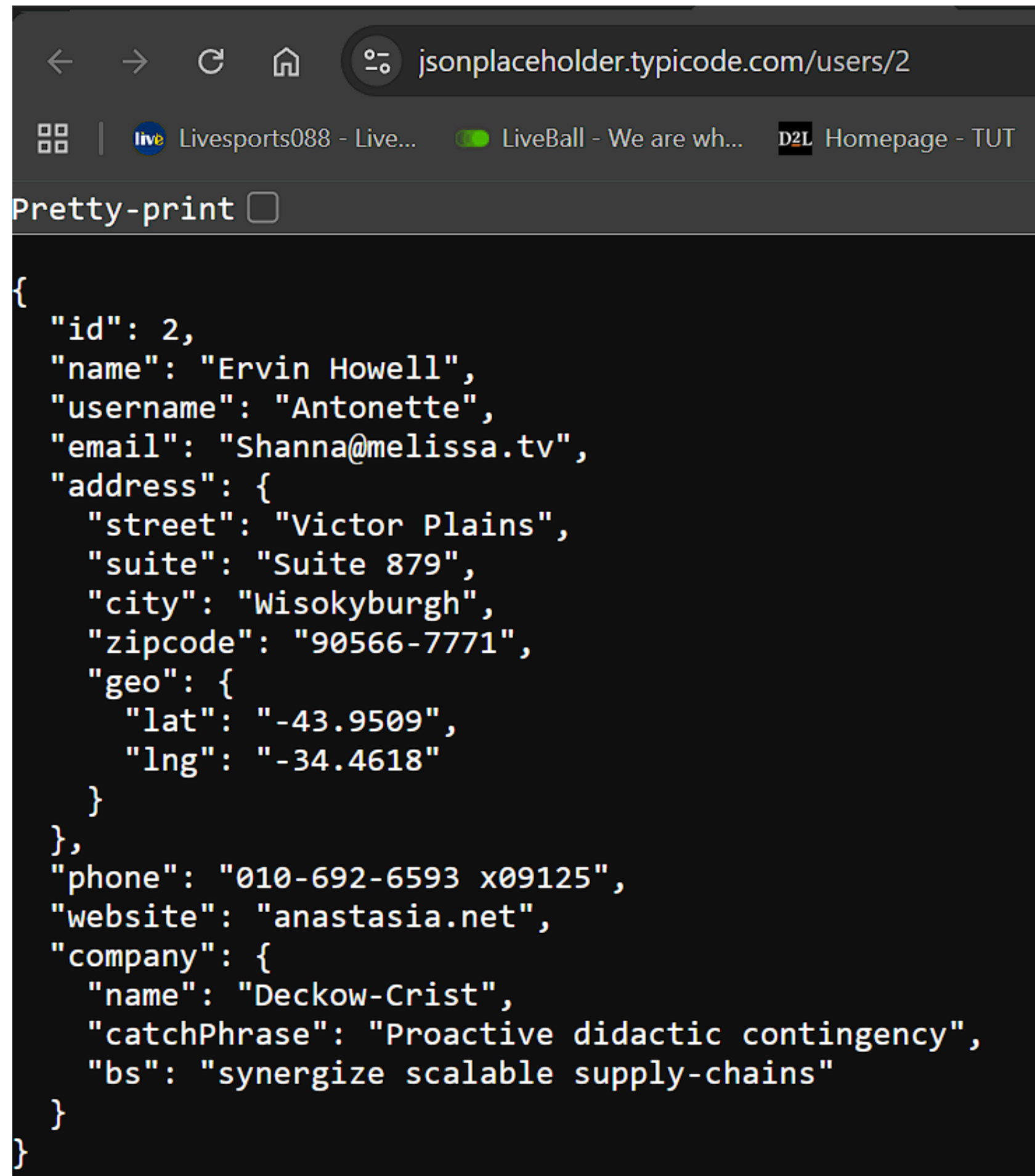
**Figure 2: Accessing User 1's sensitive profile data (IDOR vulnerability).**



The screenshot shows a web browser window with the address bar displaying `jsonplaceholder.typicode.com/users/1`. The browser's tab bar shows three tabs: "Livesports088 - Live...", "LiveBall - We are wh...", and "Homepage - TUT". Below the address bar, there is a "Pretty-print" checkbox which is currently unchecked. The main content area of the browser displays a JSON object representing user 1's profile data. The JSON is formatted with indentation and line breaks. The data includes fields for id, name, username, email, address (with street, suite, city, zipcode, and geo coordinates), phone, website, and company details (name, catchPhrase, and bs).

```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "suite": "Apt. 556",
    "city": "Gwenborough",
    "zipcode": "92998-3874",
    "geo": {
      "lat": "-37.3159",
      "lng": "81.1496"
    }
  },
  "phone": "1-770-736-8031 x56442",
  "website": "hildegard.org",
  "company": {
    "name": "Romaguera-Crona",
    "catchPhrase": "Multi-layered client-server neural-net",
    "bs": "harness real-time e-markets"
  }
}
```

**Figure 3: Accessing User 2's profile data simply by changing the URL ID parameter (IDOR).**

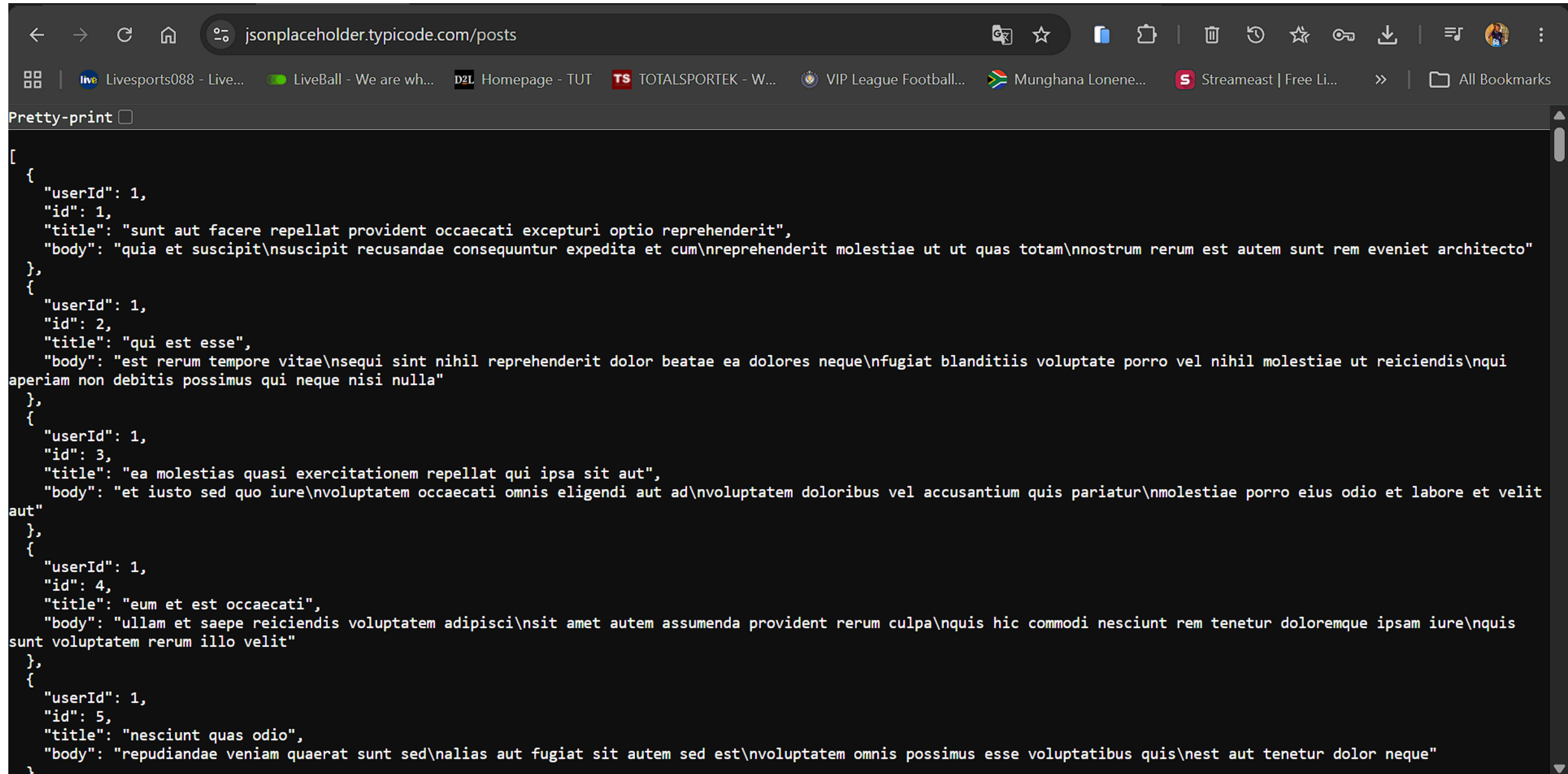


The image shows a web browser window with the address bar displaying `jsonplaceholder.typicode.com/users/2`. The browser's tab bar shows three tabs: "Livesports088 - Live...", "LiveBall - We are wh...", and "Homepage - TUT". Below the address bar, there is a "Pretty-print" checkbox. The main content area displays a JSON object representing the profile data for User 2. The JSON is formatted with indentation and line breaks. The data includes fields for id, name, username, email, address (with street, suite, city, zipcode, and geo coordinates), phone, website, and company details.

```
{
  "id": 2,
  "name": "Ervin Howell",
  "username": "Antonette",
  "email": "Shanna@melissa.tv",
  "address": {
    "street": "Victor Plains",
    "suite": "Suite 879",
    "city": "Wisokyburgh",
    "zipcode": "90566-7771",
    "geo": {
      "lat": "-43.9509",
      "lng": "-34.4618"
    }
  },
  "phone": "010-692-6593 x09125",
  "website": "anastasia.net",
  "company": {
    "name": "Deckow-Crist",
    "catchPhrase": "Proactive didactic contingency",
    "bs": "synergize scalable supply-chains"
  }
}
```



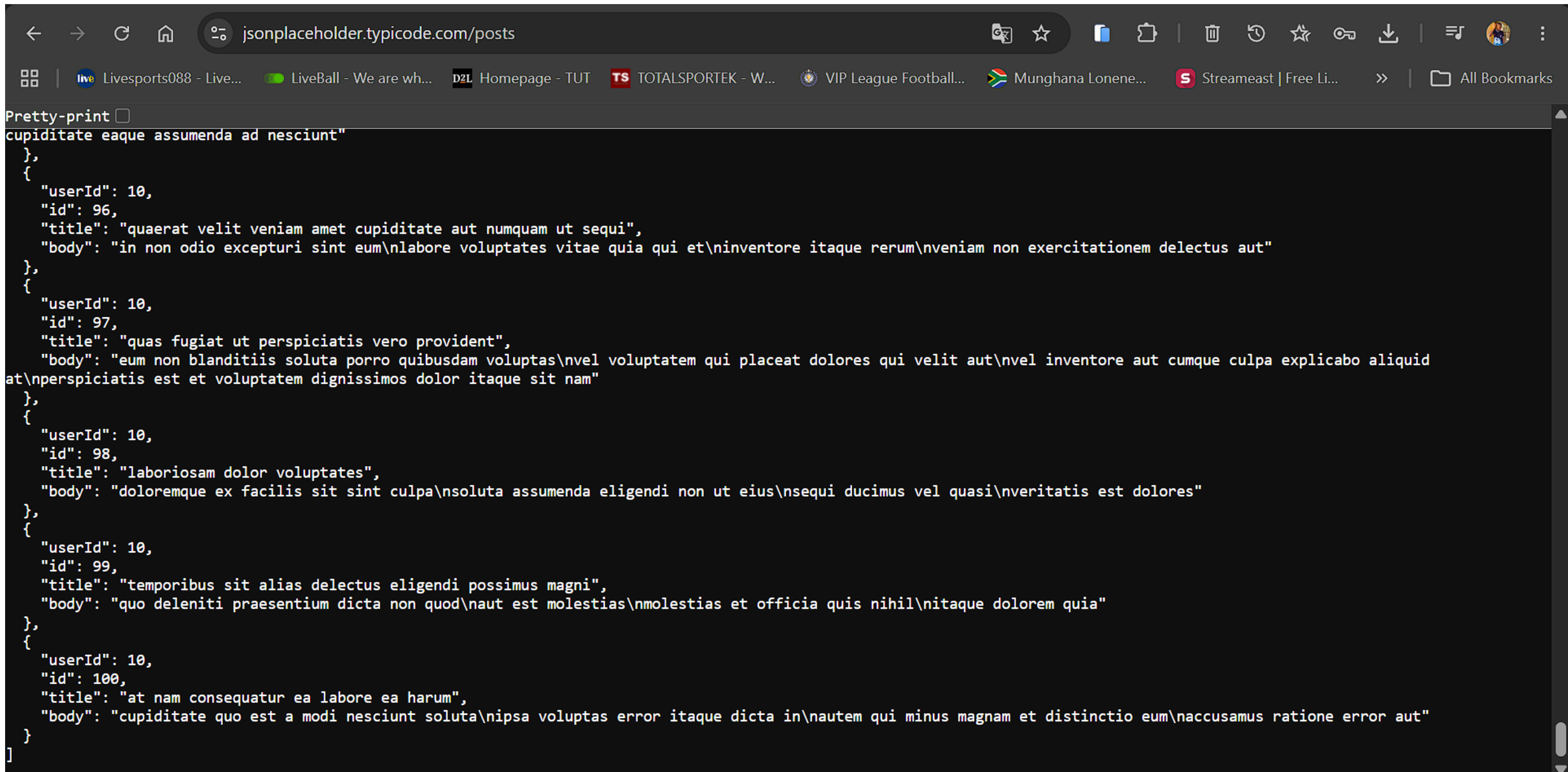
## Figure 4: Excessive Data Exposure - API dumps 100 posts in a single response with no pagination.



The screenshot shows a web browser window with the address bar displaying `jsonplaceholder.typicode.com/posts`. The browser's bookmark bar is visible, showing various sports-related links. The main content area displays a JSON response, which is a large array of 100 post objects. The first few objects are visible, showing fields like `userId`, `id`, `title`, and `body`. The `body` field contains long, repetitive Latin text, demonstrating the excessive data exposure.

```
[
  {
    "userId": 1,
    "id": 1,
    "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
    "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
  },
  {
    "userId": 1,
    "id": 2,
    "title": "qui est esse",
    "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
  },
  {
    "userId": 1,
    "id": 3,
    "title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",
    "body": "et iusto sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel accusantium quis pariatur\nmolestiae porro eius odio et labore et velit aut"
  },
  {
    "userId": 1,
    "id": 4,
    "title": "eum et est occaecati",
    "body": "ullam et saepe reiciendis voluptatem adipisci\nsit amet autem assumenda provident rerum culpa\nquis hic commodi nesciunt rem tenetur doloremque ipsam iure\nquis sunt voluptatem rerum illo velit"
  },
  {
    "userId": 1,
    "id": 5,
    "title": "nesciunt quas odio",
    "body": "repudiandae veniam quaerat sunt sed\nalias aut fugiat sit autem sed est\nvoluptatem omnis possimus esse voluptatibus quis\nest aut tenetur dolor neque"
  },
  ...
]
```

# MORE POSTS



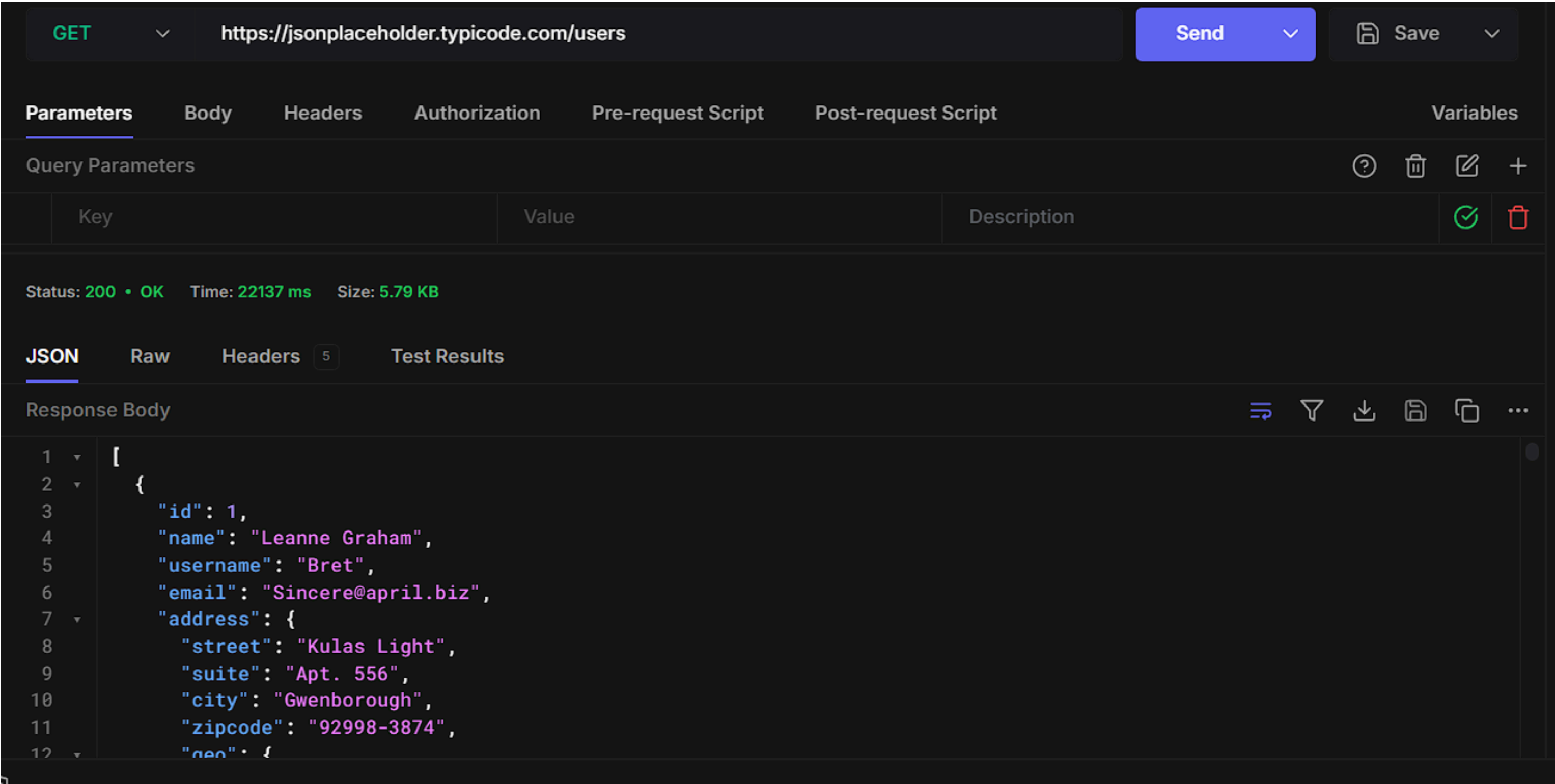
The screenshot shows a web browser window with the address bar displaying `jsonplaceholder.typicode.com/posts`. The browser's bookmark bar is visible, showing various sports-related sites. Below the browser window, a code editor displays a JSON array of four post objects. The first object has an id of 96, and the others have ids 97, 98, and 99. Each object contains a user id (all 10), a title, and a body of text.

```

Pretty-print ☐
[
  {
    "userId": 10,
    "id": 96,
    "title": "quaerat velit veniam amet cupiditate aut numquam ut sequi",
    "body": "in non odio excepturi sint eum\nlabore voluptates vitae quia qui et\ninventore itaque rerum\nveniam non exercitationem delectus aut"
  },
  {
    "userId": 10,
    "id": 97,
    "title": "quas fugiat ut perspiciatis vero provident",
    "body": "eum non blanditiis soluta porro quibusdam voluptas\nvel voluptatem qui placeat dolores qui velit aut\nvel inventore aut cumque culpa explicabo aliquid at\nperspiciatis est et voluptatem dignissimos dolor itaque sit nam"
  },
  {
    "userId": 10,
    "id": 98,
    "title": "laboriosam dolor voluptates",
    "body": "doloremque ex facilis sit sint culpa\nsoluta assumenda eligendi non ut eius\nsequi ducimus vel quasi\nveritatis est dolores"
  },
  {
    "userId": 10,
    "id": 99,
    "title": "temporibus sit alias delectus eligendi possimus magni",
    "body": "quo deleniti praesentium dicta non quod\naut est molestias\nmolestias et officia quis nihil\nitaque dolorem quia"
  }
]

```

Figure 5: API request made using Hoppscotch (Postman alternative) to GET /users. The response returns user data without requiring an API key.



**Figure 6: Request to GET /users/2 showing that any user ID can be accessed without authorization (IDOR vulnerability).**

The screenshot displays the Hoppscotch web client interface. At the top, the browser address bar shows 'hoppscotch.io'. Below it, the application header includes the 'HOPPSCOTCH' logo and a search bar. The main workspace shows a single request configuration for a GET method to the URL 'https://jsonplaceholder.typicode.com/users'. The 'Send' button is highlighted in blue. Below the URL bar, tabs for 'Parameters', 'Body', 'Headers', 'Authorization', 'Pre-request Script', 'Post-request Script', and 'Variables' are visible. The 'Parameters' tab is active, showing a table for 'Query Parameters' with columns for 'Key', 'Value', and 'Description'. Below this, the response status is shown as 'Status: 200 • OK' with a time of '22137 ms' and a size of '5.79 KB'. The 'JSON' tab is selected, displaying the response body as a JSON object for a user with ID 2. The JSON is formatted with syntax highlighting and line numbers on the left.

```

26  "id": 2,
27  "name": "Ervin Howell",
28  "username": "Antonette",
29  "email": "Shanna@melissa.tv",
30  "address": {
31    "street": "Victor Plains",
32    "suite": "Suite 879",
33    "city": "Wisokyburgh",
34    "zipcode": "90566-7771",
35    "geo": {
36      "lat": "-43.9509",
37      "lng": "-34.4618"

```



**Figure 7: The /users endpoint returns a complete list of all users (including full names, emails, and addresses) in a single request, excessive data exposure.**

The screenshot displays a web browser window with the address bar showing `jsonplaceholder.typicode.com/users`. The browser's developer tools are open, showing the 'Network' tab. The left pane displays a 'Pretty-print' view of the JSON response from the `/users` endpoint. The right pane shows the 'Network' tab with a list of requests, including `users`, `popups-script.js`, and `favicon.ico`. The 'Response' tab for the `users` request shows the full JSON data for the second user.

```
[
  {
    "id": 1,
    "name": "Leanne Graham",
    "username": "Bret",
    "email": "Sincere@april.biz",
    "address": {
      "street": "Kulas Light",
      "suite": "Apt. 556",
      "city": "Gwenborough",
      "zipcode": "92998-3874",
      "geo": {
        "lat": "-37.3159",
        "lng": "81.1496"
      }
    },
    "phone": "1-770-736-8031 x56442",
    "website": "hildegard.org",
    "company": {
      "name": "Romaguera-Crona",
      "catchPhrase": "Multi-layered client-server neural-net",
      "bs": "harness real-time e-markets"
    }
  },
  {
    "id": 2,
    "name": "Ervin Howell",
    "username": "Antonette",
    "email": "Shanna@melissa.tv",
    "address": {
      "street": "Victor Plains",
      "suite": "Suite 879",
      "city": "Wisokyburgh",
      "zipcode": "90566-7771",
      "geo": {
        "lat": "-43.9509",
        "lng": "-34.4618"
      }
    },
    "phone": "010-692-6593 x09125",
    "website": "anastasia.net",
    "company": {
      "name": "Deckow-Crist",
      "catchPhrase": "Proactive didactic contingency",
      "bs": "synergize scalable supply-chains"
    }
  }
]
```

The 'Network' tab shows the following requests:

| Name             | Size | Time   |
|------------------|------|--------|
| users            | 26   | 50 ms  |
| popups-script.js | 27   | 100 ms |
| favicon.ico      | 28   | 150 ms |

The 'Response' tab for the `users` request shows the following JSON data:

```
{
  "id": 2,
  "name": "Ervin Howell",
  "username": "Antonette",
  "email": "Shanna@melissa.tv",
  "address": {
    "street": "Victor Plains",
    "suite": "Suite 879",
    "city": "Wisokyburgh",
    "zipcode": "90566-7771",
    "geo": {
      "lat": "-43.9509",
      "lng": "-34.4618"
    }
  },
  "phone": "010-692-6593 x09125",
  "website": "anastasia.net",
  "company": {
    "name": "Deckow-Crist",
    "catchPhrase": "Proactive didactic contingency",
    "bs": "synergize scalable supply-chains"
  }
}
```

# Figure 8: Response headers showing X-Powered-By: Express and Server: heroku. This exposes the underlying technology stack to attackers, aiding reconnaissance.

The screenshot shows a web browser at the URL `jsonplaceholder.typicode.com/users`. The left sidebar displays a 'Pretty-print' JSON response for a user. The main content area shows the 'Network' tab with a list of requests: `users`, `popups-script.js`, and `favicon.ico`. The `users` request is selected, and its headers are displayed in the right pane.

**JSON Response (Pretty-print):**

```
[
  {
    "id": 1,
    "name": "Leanne
Graham",
    "username": "Bret",
    "email":
"Sincere@april.biz",
    "address": {
      "street": "Kulas
Light",
      "suite": "Apt. 556",
      "city":
"Gwenborough",
      "zipcode": "92998-
3874",
      "geo": {
        "lat": "-37.3159",
        "lng": "81.1496"
      }
    },
    "phone": "1-770-736-
8031 x56442",
    "website":
"hildegard.org",
    "company": {
      "name": "Romaguera-
Crona",
      "catchPhrase":
"Multi-layered client-
server neural-net",
      "bs": "harness real-
time e-markets"
    }
  }
]
```

**Network Headers for `users`:**

| Name                   | Value                                                                                                                                                                                                                       |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Priority               | u=0,i                                                                                                                                                                                                                       |
| Report-To              | { "group": "heroku-nel", "endpoints": [ { "url": "https://nel.heroku.com/reports?s=PBJOQYTmpF%2FWUsyyAhG8UVS7zHD7uZWJkKz7Dwfb70%3D\u0026sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d\u0026ts=1777075221" } ], "max_age": 3600 } |
| Reporting-Endpoints    | heroku-nel="https://nel.heroku.com/reports?s=PBJOQYTmpF%2FWUsyyAhG8UVS7zHD7uZWJkKz7Dwfb70%3D\u0026sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d&ts=1777075221"                                                                   |
| Server                 | cloudflare                                                                                                                                                                                                                  |
| Server-Timing          | cfCacheStatus;desc="HIT"                                                                                                                                                                                                    |
| Server-Timing          | cfEdge;dur=3,cfOrigin;dur=0                                                                                                                                                                                                 |
| Server-Timing          | cfExtPri                                                                                                                                                                                                                    |
| Vary                   | Origin, Accept-Encoding                                                                                                                                                                                                     |
| Via                    | 2.0 heroku-router                                                                                                                                                                                                           |
| X-Content-Type-Options | nosniff                                                                                                                                                                                                                     |
| X-Powered-By           | Express                                                                                                                                                                                                                     |
| X-Ratelimit-Limit      | 1000                                                                                                                                                                                                                        |
| X-Ratelimit-Remaining  | 997                                                                                                                                                                                                                         |
| X-Ratelimit-Reset      | 1777075245                                                                                                                                                                                                                  |

At the bottom of the network pane, it shows: 3 requests | 16.0 kB transferred | 21.2 kB res.

# Conclusion

The API security risk analysis identified critical vulnerabilities in the JSONPlaceholder API, including unauthenticated access, broken authorization (IDOR), excessive data exposure, and server information disclosure. These issues are common in poorly secured SaaS applications and can lead to significant data breaches if implemented in production environments.

By implementing token-based authentication, strict access controls, pagination, and rate limiting, organizations can substantially improve their API security posture and protect sensitive customer data.

Regular API security audits are essential for modern businesses to stay ahead of evolving threats.